

LSINF2275 – Data mining and decision making

Markov Decision Processes and Reinforcement Learning

Project guidelines

Professor: Marco Saerens, UCLouvain

Teaching Assistants: Pierre Leleux and Nicolas Szelagowski

Objective

The objective of this project is to put into practice some of the techniques introduced in the data mining and decision making lectures. This is done through the study of a practical case which requires the use of a scientific programming language (Python) and packages related to the statistical processing of data. For the *first part* of the project, the work aims at applying algorithms (mainly the value iteration) solving “Markov decision processes” in the framework of a Snakes and Ladders game. The *second part* is less constrained and more creative; there are no strict guidelines for it. You will be free to study another interesting practical application or a theoretical extension of Markov decision processes or reinforcement learning, on the Snakes and Ladders game, or another application of your choice.

Part I

This is the first part of the project (there is no deadline for the first part, but we suggest you finishing it around week 7-8).

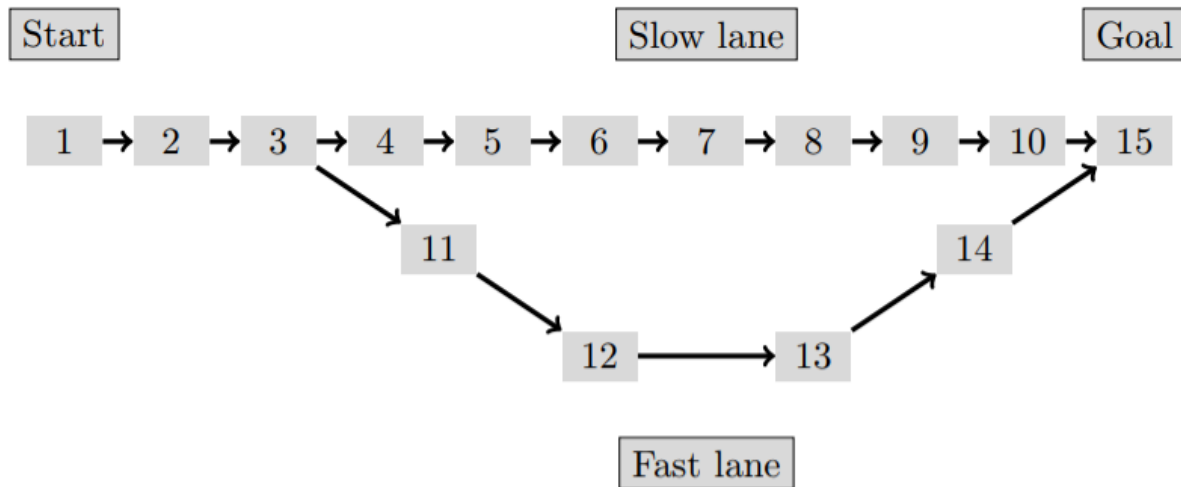
1 Problem statement

The realization of the project will be carried out in groups of maximum 3 students (2 students only if 3 is not possible). You are asked to solve the following problem.

Let us assume that you are playing a Snakes and Ladders game which is made of 15 squares. Square number 1 is the initial square (start) and square 15 is the winning square (goal). If you reach square 15, you win. The board can contain traps that slow down your progression if triggered. A schematic representation of this game is shown below.

At any time during the game, to move forward, you can choose between three dice:

- The “security” dice which can only take two possible values: 0 or 1. It allows you to move forward by 0 or 1 square, with a probability of $1/2$. With that dice, you are invincible, which means that you ignore the presence of traps when playing with the security dice – you are immune to traps.
- The “normal” dice, which allows the player to move by 0, 1 or 2 squares with a probability of $1/3$. If you land on a trapped square using this dice, you have 50 % chance of triggering the trap.



- The “risky” dice which allows the player to move by 0, 1, 2 or 3 squares with a probability of $1/4$. However, when playing with the risky dice, you automatically trigger any trap you land on (100 % trigger probability).

At square 3, there is a junction with two possible paths. If the player just passes through square 3 without stopping, they continue to square 4 or beyond (5, 6, etc), as if the other path does not exist. Conversely, if the player stops on square 3, they will have, on the following turn, an equal probability of taking the slow or the fast lane. For instance, for the security dice, if the player is on square 3 and the result is 0, the player stays on square 3 with probability 1. If the result is 1, they reach square 4 or 11, each with probability $1/2$. Both paths ultimately lead to the final, goal, square.

You should also define “trap” squares on the game, that can be triggered when stopping exactly on that square. Recall that the player is only exposed to traps when drawing the normal or the risky dice. There are different types of trap, each one has its own effect:

- Type 1 – Restart: Immediately teleports the player back to the first square.
- Type 2 – Penalty: Immediately teleports the player 3 squares backwards.
- Type 3 – Prison: The player must wait one turn before playing again.
- Type 4 – Mystery: Randomly applies the effect of one the three previous traps, with an equal probability.

Note that the first and final squares cannot be trapped.

We ask you to compute the optimal strategy, i.e., for each square, which dice should be used to reach the goal square in a minimal number of turns, on average (the cost represents the number of turns to reach the goal square). You should determine this solution in two different scenarios:

- You should exactly stop on the arrival, goal, square to win. The game board is designed as a circle, which means that, if you overstep the last square, you have to restart from the 1st square.

- You win as soon as you land on or overstep the final square (i.e. if you are on square 15 or further).

2 Implementation

You are asked to implement, in *Python* 3, the following function

```
markovDecision(layout,circle)
```

This function launches the Markov Decision Process algorithm to determine the optimal strategy regarding the choice of the dice in the Snakes and Ladders game, using the “value iteration” method.

Inputs:

- **layout**: a vector of type `numpy.ndarray` that represents the layout of the game, containing 15 values representing the 15 squares of the Snakes and Ladders game:

`layout[i] = 0` if it is an ordinary square
`= 1` if it is a “restart” trap (go back to square 1)
`= 2` if it is a “penalty” trap (go back 3 steps)
`= 3` if it is a “prison” trap (skip next turn)
`= 4` if it is a “mystery” trap (random effect among the three previous)

- **circle**: a boolean variable (type `bool`), indicating if the player must land exactly on the final, goal, square 15 to win (`circle = True`) or still wins by overstepping the final square (`circle = False`).

Output: Your function `markovDecision` is expected to return a type `list` containing the two vectors `[Expec,Dice]`:

- **Expec**: a vector of type `numpy.ndarray` containing the expected cost (= number of turns) associated to the 14 squares of the game, excluding the goal square. The vector starts at index 0 (corresponding to square 1 on the example) and ends at index 13 (square 14).
- **Dice**: a vector of type `numpy.ndarray` containing the choice of the dice to use for each of the 14 squares of the game (1 for “security” dice, 2 for “normal” dice and 3 for “risky”), excluding the goal square. Again, the vector starts at index 0 (square 1) and ends at index 13 (square 14).

Important: Beware that the correctness and efficiency of your function will be graded by an automated script that will execute your function several times using different board layouts to check if your function always returns the correct policy and expected costs. It is therefore important that you strictly comply with the syntax of the function (name, inputs and outputs). Your function `markovDecision` must be in a file named “`part-I.py`”, not a Jupyter notebook.

In addition, it is also interesting to compare this strategy with other (sub-optimal) strategies, e.g. the use of dice 1, dice 2 or dice 3 only, a mixed random strategy, or a purely random choice,

etc. Launch/simulate a large number of games and compare empirically the performance of each of these strategies with the optimal strategy (or policy) obtained by value iteration. You should then report (i) the *theoretical expected cost*, obtained by value iteration, and compare it with the *empirical average cost* when playing (simulating) a large number of games with the optimal strategy, and (ii) compare the empirical average cost obtained by the optimal strategy with the other (sub-optimal) strategies mentioned before (dice 1 only, etc). You could test this with a few different configurations of traps, as well as for the different scenarios (circle or not), depending on your time.

3 Practicalities

Consider the following practical details concerning the implementation:

1. Traps can be triggered when drawing a 0 from the dice. For instance, if you use the normal or the risky dice and get a 0, while standing on a trapped square, you (may) trigger the trap of the square you are standing on.
2. There is no “cascade triggering” of traps: if you trigger a trap that teleports you (type 2 or 4), you ignore the potential presence of a trap on the square you got teleported to. Note that, however, if you play with the normal or risky dice, you may still trigger the trap next turn by drawing a 0 (see previous point).
3. Penalty traps (type 2) situated too close to the beginning (on square 2 or 3), that cannot make the player move 3 squares backwards, simply teleport the player on the first square like a restart trap (type 1) would. This rule also applies in the case of **circular** boards (`circle = True`).

Part II

This is the second part of the project (up to week 13).

4 Second part – additional investigations

As for Part I, you will be working in groups of 3 students exactly – using the same groups as for Part I.

This part is less constrained and more creative; there are no strict guidelines for it. Besides the basics of Markov decision processes, it is clear that it would also be interesting to investigate Q -learning or function approximation (based on features instead of states), as well as other reinforcement learning techniques on the Snakes and Ladders game, and study to which extend these algorithms are competitive with value iteration (both in terms of computation time and average cost to reach the goal state).

Alternatively, it would also be interesting to tackle a different application, for instance related to another game or to artificial intelligence applications, etc.

In this second assignment, you are asked to study an interesting practical application of Markov decision processes (MDP), reinforcement learning (RL), bandit problems (BP), or a theoretical extension of the basic algorithms together with an empirical validation and comparison with the value iteration algorithm, if applicable. Good starting points are the books cited

in the bibliography, especially Sutton & Barto and Kochenderfer et al., as well as, of course, the internet. So, do not hesitate to be creative! Unlike the first part, for this second part, feel free to use any Python library or code you want (provided it is allowed of course).

The topic is very open – more concretely, here are some potential ideas.

- Find (either in the literature, or create a new one) a nice practical application of MDP, RL or BP, for instance a game; describe it, implement it and validate it empirically. Or tackle an ambitious practical application on OpenAI, or any other source, with MDP or RL.
- Investigate a theoretical extension of MDP, RL or BP. For instance, Monte-Carlo methods in RL, together with importance sampling. Describe briefly the theory behind the method and validate it on a practical example (possibly the Snakes and Ladders game).
- Investigate and compare standard RL algorithms, like Q-learning, on the Snakes and Ladders game. Also describe briefly the theory behind the methods and compare the obtained results with the optimal policy provided by the value iteration algorithm.
- Investigate a theoretical extension of MDP, RL or BP. For instance, n -step SARSA methods in RL. Describe briefly the theory behind the method and validate it on a practical application.
- Investigate a theoretical extension of MDP, RL or BP. For instance, approximate RL dealing with systems with a large (exponential) number of states, based on a simple regression or a neural network. Describe briefly the theory behind the method and validate it on a practical application.
- Investigate a theoretical extension of MDP. For instance, an exploration strategy like, e.g., entropy-regularized MDPs. Describe briefly the theory behind the method and validate it on a practical application.
- Investigate some potential solutions to deal with partially observable Markov decision processes (POMDP) and compare them empirically on a practical application.
- Etc...

Thus, this second part will be based on scientific papers, books or external libraries, and should contain both a theoretical contribution (description of the method) as well as an empirical validation. The balance between the two components depends on your wish and expectation: the orientation of the work can be more theoretical or more practical. If, for instance, you decide to face a complex application, you can limit yourself to a description of the basic algorithms you used (MDP, RL or BP). Conversely, if you decide to study a theoretical extension, the practical application can just be the “Snakes and Ladders” game investigated in the first project.

5 Final Report

Please do not forget to mention your affiliation on the cover page, together with your name (SINF, INFO, MAP, STAT, BIR, DATS, etc) and group number. You are asked to write a report (PDF) in English of maximum 6 pages for Part I and 6 pages for Part II. The final, complete, report (including both the report for Part I + the report for Part II) should be no

longer than 12 pages (everything included, except bibliography). This final report as well as the code¹ (everything zipped together) must be uploaded on Moodle on May 20, 23:55 (the deadline is strict²), for each group. Recall that the report is worth 8 points on 20 and the final exam 12 points on 20.

This complete report must have a professional look & feel, like a scientific or a technical report. Therefore, your report can integrate plots (only if pertinent), but no screenshot of equations and/or screenshot of code outputs. Do not forget to provide references for your sources of information³ and be consistent in your notations. A list of common pitfalls when writing such a report is provided in the Appendix.

Your report for **Part I** must at least contain:

- a short introduction;
- a brief theoretical explanation of the method used to determine the optimal strategy regarding the choice of the dice;
- a short description of your implementation⁴;
- for various trap layouts (trap squares at relevant places), comparisons between theoretical and empirical results for the optimal strategy as well as between the optimal and suboptimal strategies. Analyze and discuss your results.
- a conclusion.

Your report for **Part II** must at least contain:

- a short introduction;
- a presentation of the chosen application (unless you decided to keep on working on the Snakes and Ladders);
- a theoretical presentation of the model, and its relevance with respect to your application;
- an analysis and discussion of the results;
- a conclusion.

Your project will be evaluated on the following aspects:

- The quality of your report and code;
- The amount of experimental work (including potential additional experiments);
- The correctness and efficiency of your function `markovDecision`.

¹Feel free to separate the code of part I and part II into two different folders for clarity.

²Moodle will block any late submission, so be mindful and avoid uploading your project at the last minute.

³A good reference available online is the book of Sutton and Barto (2018) “Reinforcement learning: an introduction”, second edition.

⁴Do not go into details about your code itself. If you still wish to provide details about the code and the functions of your implementation, do not hesitate to integrate a ReadMe file in your zip, but your report should not focus on the code.

6 References

- Bertsekas & Tsitsiklis (1996) “Neuro-dynamic programming”. Athena Scientific.
- Brandimarte (2021) “From shortest paths to reinforcement learning: a Matlab-based tutorial on dynamic programming”. Springer.
- Kochenderfer, Wheeler & Wray (2022) “Algorithms for decision making”. MIT Press.
- Powell (2011) “Approximate dynamic programming”, 2nd ed.”. Wiley.
- Powell & Ryzhov (2012) “Optimal learning”. Wiley.
- Puterman (1994) “Markov decision processes”. Wiley.
- Russell & Norvig (2022) “Artificial intelligence: a modern approach”, 4th ed. Prentice-Hall.
- Plaat (2020) “Learning to play: reinforcement learning and games”. Springer.
- Plaat (2022) “Deep reinforcement learning”. Springer.
- Sutton & Barto (2018) “Reinforcement learning: an introduction”, 2nd ed.”. MIT Press.
- Szepesvari (2010) “Algorithms for reinforcement learning”. Morgan and Claypool Publishers.

Appendix: Common pitfalls

For your information, here are some common issues that frequently appeared in the last years corrected reports. This list is for all types of reports, asked in several different courses; therefore some of these remarks might not apply in your special case.

- Please follow strictly the instructions. For instance, if we ask a theoretical description of the models, do not forget to include it. If it is asked, it means that we assign a mark on it.
- The report should be written in the same way as a scientific report or paper (we recommend using Latex) and the submitted version must be in PDF.
- Also, if we describe the structure of the procedure(s) or function to be implemented in the code (signature, etc), or the name of the files that have to be delivered, in the instructions, please follow precisely these guidelines. We usually write a program that automatically tests your procedure or reads your file; if the guidelines are not applied correctly, it crashes. It results in a waste of time and penalties.
- No screenshot of equations or algorithms from the web or from any other source. This is considered as non-professional in our community. Also, add a reference to the sources in the caption or the text if you copy a figure.
- In the theoretical part, be coherent in your mathematical notation (use consistently the same notation and style for the same quantity in all the equations and the text).

- Use the same mathematical notation in the text and in the equations (if scalar variables are in italic in the equations, they should also be in italic in the text when you refer to these).
- Clearly explain the notation and the intuition behind each main equation.
- The look & feel of the report must be professional, like a scientific or a technical report.
- Do not forget the references / sources in the explanations – any claim must be justified by references.
- Do not cite fancy web sites in the bibliography if there are good reference textbooks or scientific papers covering the material. Web sites are changing all the time and are themselves copying their material from more reliable sources like technical books, technical or scientific reports. Moreover, a good textbook uses the same notation and conventions all along which is more convenient.
- Use exactly the same format and structure for all your bibliographical references
- Clearly explain the validation methodology when evaluating the models or comparing them. The procedure should be sufficiently clear and detailed so that it could be reproduced by the reader.
- Provide a clear presentation and discussion of the obtained experimental results.
- For the tables and plots of the results, indicate the quantities that are reported as well as their units in the caption or/and the figure.
- The explanation of the detailed code (with copy/paste of code examples) in the report should not be too long (the code is anyway provided). The algorithm(s) could be explained in detail, but then in the theoretical section.
- Do not forget to comment your code.

Good work!
